Assignment - 6

Name: Naga Rishik Reddy

Course: AI Assisted coding

Hall ticket no: 2303A51089

Task Description #1 (Loops – Automorphic Numbers in a Range)

• Task: Prompt AI to generate a function that displays all Automorphic numbers between 1 and 1000 using a for loop.

• Instructions:

o Get AI-generated code to list Automorphic numbers using a for loop Analyze the correctness and efficiency of the generated logic.

o Ask AI to regenerate using a while loop and compare both implementations.

Expected Output #1:

• Correct implementation that lists Automorphic numbers using both loop types, with explanation.

```python
#task 1:
#generate a function that displays all automorphic numbers between 1 to 1000 using a for loop
def is_automorphic(num):
    square = num * num
    return str(square).endswith(str(num))
for i in range(1, 1001):
    if is_automorphic(i):
        print(f"{i} is an automorphic number.")

print("\n")
#regenrate using a while loop and compare both implementations
def is_automorphic(num):
    square = num * num
    return str(square).endswith(str(num))
i = 1
while i <= 1000:
    if is_automorphic(i):
        print(f"{i} is an automorphic number.")
    i += 1
#by using time module compare the execution time of both implementations
import time
start_for = time.time()
for i in range(1, 1001):
    if is_automorphic(i):
        pass
end_for = time.time()
```

output:

```python
        pass
end_for = time.time()

start_while = time.time()
i = 1
while i <= 1000:
    if is_automorphic(i):
        pass
    i += 1
end_while = time.time()
print(f"for loop execution time: {end_for - start_for} seconds")
print(f"while loop execution time: {end_while - start_while} seconds")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
1 is an automorphic number.
5 is an automorphic number.
5 is an automorphic number.
6 is an automorphic number.
6 is an automorphic number.
25 is an automorphic number.
76 is an automorphic number.
76 is an automorphic number.
376 is an automorphic number.
625 is an automorphic number.
for loop execution time: 0.0007481575012207031 seconds
for loop execution time: 0.0007481575012207031 seconds
while loop execution time: 0.0008666515350341797 seconds
PS C:\Users\Rishi\Documents\Naga Rishik\AI assisted>
```

Activate Windows
Go to Settings to activate

Task Description #2 (Conditional Statements – Online Shopping Feedback Classification)

• Task: Ask AI to write nested if-elif-else conditions to classify online shopping feedback as Positive, Neutral, or Negative based on a numerical rating (1–5).

• Instructions:

o Generate initial code using nested if-elif-else.

o Analyze correctness and readability.

o Ask AI to rewrite using dictionary-based or match-case structure.

Expected Output #2:

• Feedback classification function with explanation and an alternative approach

```python
#task 2:
#write nested if-else conditions to classify online shopping feedback as positive, neutral, or Negative bas
rating = 4 #example rating
if rating >= 4:
    feedback = "Positive"
elif rating == 3:
    feedback = "Neutral"
else:
    feedback = "Negative"
print(f"Rating: {rating}, Feedback: {feedback}") #output: the feedback is:positive
#rewrite using dictionary-based or match-case structure
feedback_dict = {
    5: "Positive",
    4: "Positive",
    3: "Neutral",
    2: "Negative",
    1: "Negative"
}
feedback = feedback_dict.get(rating, "Invalid rating")
print(f"Rating: {rating}, Feedback: {feedback}") #output: the feedback is:positive
#analyze correctness and readability
#the first implementation using nested if-elif-else is straightforward and esay to understand for simple cc
#the second implementation using a dictionary provides a cleaner and more scalable solution, especially if
#both implementations are correct, but the dictionary-based approach is generally more maintainable and eas
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

while loop execution time: 0.0008666515350341797 seconds
PS C:\Users\Rishi\Documents\Naga Rishik\AI assisted> & C:\Users\Rishi\anaconda\python.exe "c:/Users/Rishi/Documents/Naga Ri
shik/AI assisted/ass6.py"
Rating: 4, Feedback: Positive
Rating: 4, Feedback: Positive
PS C:\Users\Rishi\Documents\Naga Rishik\AI assisted>
```

Activate Windows
Go to Settings to activate

Task 3: Statistical_operations

Define a function named statistical_operations(tuple_num) that performs the following statistical operations on a tuple of numbers:

• Minimum, Maximum

• Mean, Median, Mode

• Variance, Standard Deviation

While writing the function, observe the code suggestions provided by GitHub

Copilot.Make decisions to accept, reject, or modify the suggestions based on their relevance and correctness

```python
ass6.py > ...
#task 3:
#define a function named statistical_operations(tuple_num) that perfoms the following statistical operatior
#minimum,maximum
#mean,median,mode
#standard deviation,variance
import statistics
def statistical_operations(tuple_num):
    minimum = min(tuple_num)
    maximum = max(tuple_num)
    mean = statistics.mean(tuple_num)
    median = statistics.median(tuple_num)
    mode = statistics.mode(tuple_num)
    stdev = statistics.stdev(tuple_num)
    variance = statistics.variance(tuple_num)

    return {
        'minimum': minimum,
        'maximum': maximum,
        'mean': mean,
        'median': median,
        'mode': mode,
        'standard_deviation': stdev,
        'variance': variance
    }
# Example usage
data=(1, 2, 2, 3, 4)
result = statistical_operations(data)
for key, value in result.items():
```

output:

```python
for key, value in result.items():
    print(f"{key}: {value}")
#output:
#minimum: 1
#maximum: 6
#mean: 2.4
#median: 2
#mode: 2
#standard_deviation: 1.140175425099138
#variance: 1.3
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
minimum: 1
maximum: 4
mean: 2.4
mean: 2.4
median: 2
median: 2
mode: 2
standard_deviation: 1.140175425099138
mode: 2
standard_deviation: 1.140175425099138
standard_deviation: 1.140175425099138
variance: 1.3
```

Activate Windows
Go to Settings to activate W

Task 4: Teacher Profile

• Prompt: Create a class Teacher with attributes teacher_id, name, subject, and experience. Add a method to display teacher details.

Expected Output: Class with initializer, method, and object creation.

```python
#task 4:
#create a class teacher with attributes teacher_id, name, subject, and experience. add a
class teacher:
    def __init__(self, teacher_id, name, subject, experience):
        self.teacher_id = teacher_id
        self.name = name
        self.subject = subject
        self.experience = experience

    def display_details(self):
        return (f"Teacher ID: {self.teacher_id}\n"
                f"Name: {self.name}\n"
                f"Subject: {self.subject}\n"
                f"Experience: {self.experience} years")
# Example usage:
teacher1 = teacher(1, "John Doe", "Mathematics", 10)
print(teacher1.display_details())
#output:
#Teacher ID: 1
#Name: John Doe
#Subject: Mathematics
#Experience: 10 years
teacher2 = teacher(2, "Jane Smith", "English", 5)
print(teacher2.display_details())
#output:
#Teacher ID: 2
#Name: Jane Smith
#Subject: English
```

output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

Teacher ID: 1
Name: John Doe
Subject: Mathematics
Experience: 10 years
Teacher ID: 2
Name: Jane Smith
PS C:\Users\Rishi\Documents\Naga Rishik\AI assisted> & C:\Users\Rishi\anaconda\python.exe "c:/Users
/Rishi/Documents/Naga Rishik/AI assisted/ass6.py"
Teacher ID: 1
Name: John Doe
Subject: Mathematics
Experience: 10 years
Teacher ID: 2
Name: Jane Smith
Name: Jane Smith
Subject: English
Experience: 5 years
PS C:\Users\Rishi\Documents\Naga Rishik\AI assisted>

powershell
Python
```

Activate Windows
Go to Settings to activate Windo

Task Description #6 (Loops – Armstrong Numbers in a Range)

Task: Write a function using AI that finds all Armstrong numbers in a user-specified range (e.g., 1 to 1000).

Instructions:

• Use a for loop and digit power logic.

• Validate correctness by checking known Armstrong numbers (153, 370, etc.).

• Ask AI to regenerate an optimized version (using list comprehensions).

Expected Output #7:

• Python program listing Armstrong numbers in the range.

• Optimized version with explanation.

program1.py > ...

```python
#task 6:
#write a funvtion using AI that finds all Armstrong numbers in a user-specified range (e.g., 1 to 1000)
#INSTRUCTIONS:
#use only for loop and digit power logic
#validate correctness by checking known Armstrong numbers (153, 370,etc...)
#ask AI to regenerate an optimized version (using list comprehensions)

def is_armstrong(num):
    digits = [int(d) for d in str(num)]
    power = len(digits)
    sum_of_powers = sum(d ** power for d in digits)
    return sum_of_powers == num
armstrong_numbers = []
for i in range(1, 1001):
    if is_armstrong(i):
        armstrong_numbers.append(i)
print("Armstrong numbers between 1 and 1000:", armstrong_numbers)
#output: Armstrong numbers between 1 to 1000 are: [1, 2, 3, 4, 5, 6, 7, 8, 9, 153, 370, 371, 407]
#optimized version using list comprehensions
armstrong_numbers_optimized = [i for i in range(1, 1001) if is_armstrong(i)]
print("Armstrong numbers between 1 and 1000 (optimized):", armstrong_numbers_optimized)
#output: Armstrong numbers between 1 and 1000 (optimized): [1, 2, 3, 4, 5, 6, 7, 8, 9, 153, 370, 371, 407]
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Rishi\Documents\Naga Rishik\AI assisted> (C:\Users\Rishi\anaconda\Scripts\activate) ; (conda activate base)
PS C:\Users\Rishi\Documents\Naga Rishik\AI assisted> & C:\Users\Rishi\anaconda\python.exe "c:/Users/Rishi/Documents/Naga Ri
shik/AI assisted/program1.py"
Armstrong numbers between 1 and 1000: [1, 2, 3, 4, 5, 6, 7, 8, 9, 153, 370, 371, 407]
Armstrong numbers between 1 and 1000 (optimized): [1, 2, 3, 4, 5, 6, 7, 8, 9, 153, 370, 371, 407]
PS C:\Users\Rishi\Documents\Naga Rishik\AI assisted>
```

Activate Windows
Go to Settings to activate Windows.

Task Description #7 (Loops – Happy Numbers in a Range)

Task: Generate a function using AI that displays all Happy Numbers within a user-specified range (e.g., 1 to 500).

Instructions:

• Implement the logic using a loop: repeatedly replace a number with the sum of the squares of its digits until the result is either 1 (Happy Number) or enters a cycle (Not Happy).

• Validate correctness by checking known Happy Numbers (e.g., 1, 7, 10, 13, 19, 23, 28…).

• Ask AI to regenerate an optimized version (e.g., by using a set to detect cycles instead of infinite loops).

Expected Output #8:

• Python program that prints all Happy Numbers within a range.

• Optimized version using cycle detection with explanation

program1.py > is_happy_optimized

```python
#task 7:
#generate a function using AI that displays all happy numbers within a user-specified range(e.g., 1 to 500)
#implement the logic using a loop: repeatedly replace a number with the sum of the squares of its digits ur
#validate correctness by checking known happy numbers (e.g., 1, 7, 10, 13, 19, 23, 28...)
#ask ai to regenerate an optimed version (e.g., by using a set to detect cycles instead of infinite loops)
def is_happy(num):
    seen = set()
    while num != 1 and num not in seen:
        seen.add(num)
        num = sum(int(digit) ** 2 for digit in str(num))
    return num == 1
happy_numbers = []
for i in range(1, 501):
    if is_happy(i):
        happy_numbers.append(i)
print("Happy numbers between 1 and 500:", happy_numbers)
#output: happy numbers between 1 and 500: [1, 7, 10, 13, 19, 23, 28, 31, 32, 44, 49, 68, 70, 79, 82, 86, 91
#optimized version using a set to detect cycles:
def is_happy_optimized(num):
    seen = set()
    while num != 1 and num not in seen:
        seen.add(num)
        num = sum(int(digit) ** 2 for digit in str(num))
        return false
    seen.add(num)
    num = sum(int(digit) ** 2 for digit in str(num))
    return True
happy_numbers_optimized = [num for num in range(1, 501) if is_happy_optimized(num)]
print("Happy numbers between 1 and 500 (optimized):", happy_numbers_optimized)
#output: happy numbers between 1 and 500 (optimized): [1, 7, 10, 13, 19, 23, 28, 31, 32, 44, 49, 68, 70, 79
#the optimized version improves cycle detection by using a set to track seen numbers, preventing infinite
```

Activate Windows
Go to Settings to activate Windows.

output:

```
Happy numbers between 1 and 500: [1, 7, 10, 13, 19, 23, 28, 31, 32, 44, 49, 68, 70, 79, 82, 86, 91, 94, 97, 100, 103, 109,
129, 130, 133, 139, 167, 176, 188, 190, 192, 193, 203, 208, 219, 226, 230, 236, 239, 262, 263, 280, 291, 293, 301, 302, 310
, 313, 319, 320, 326, 329, 331, 338, 356, 362, 365, 367, 368, 376, 379, 383, 386, 391, 392, 397, 404, 409, 440, 446, 464, 4
69, 478, 487, 490, 496]
```

Task Description #8 (Loops – Strong Numbers in a Range)

Task: Generate a function using AI that displays all Strong Numbers (sum offactorial of digits equals the number, e.g., 145 = 1!+4!+5!) within a given range.

Instructions:

• Use loops to extract digits and calculate factorials.

• Validate with examples (1, 2, 145).

• Ask AI to regenerate an optimized version (precompute digit factorials).

Expected Output #9:

• Python program that lists Strong Numbers.

• Optimized version with explanation.

```python
#task8:
'''
genrate a function using AI that displays all strong numbers (sum of factorial of digits equala the number,
instructions:
> use loops to extract digits and calculate factorials
> validate with examples (1, 2, 145)
> ask ai to regenerate an optimized version (precompute digit factorials)
'''
import math
def is_strong(num):
    sum_of_factorials = 0
    for digit in str(num):
        sum_of_factorials += math.factorial(int(digit))
    return sum_of_factorials == num
strong_numbers = []
for i in range(1, 1001):
    if is_strong(i):
        strong_numbers.append(i)
        print("Strong numbers between 1 and 1000:", strong_numbers)
#output: optimized strong numbers between 1 and 1000: [1, 2, 145, 40585]
#optimized version with precomputed digit factorials
digit_factorials = {str(i): math.factorial(i) for i in range(10)}
def is_strong_optimized(num):
    sum_of_factorials = sum(digit_factorials[digit] for digit in str(num))
    return sum_of_factorials == num
strong_numbers_optimized = [num for num in range(1, 1001) if is_strong_optimized(num)]
print("Optimized strong numbers between 1 and 1000:", strong_numbers_optimized)
#output: optimized strong numbers between 1 and 1000: [1, 2, 145, 40585]
#the optimized version reduces redundant calculations by precomputing the factorials of digits 0-9, improvi
```

Activate Windows

output:

```
PS C:\Users\Rishi\Documents\Naga Rishik\AI assisted> (C:\Users\Rishi\anaconda\Scripts\activate) ; (conda activate base)
PS C:\Users\Rishi\Documents\Naga Rishik\AI assisted> & C:\Users\Rishi\anaconda\python.exe "c:/Users/Rishi/Documents/Naga Ri
shik/AI assisted/assignments.py"
Strong numbers between 1 and 1000: [1]
Strong numbers between 1 and 1000: [1, 2]
Strong numbers between 1 and 1000: [1, 2, 145]
Optimized strong numbers between 1 and 1000: [1, 2, 145]
PS C:\Users\Rishi\Documents\Naga Rishik\AI assisted>
```

Activate Windows
Go to Settings to activate W

Task #9 – Few-Shot Prompting for Nested Dictionary Extraction

Objective

Use few-shot prompting (2–3 examples) to instruct the AI to create a function that parses a nested dictionary representing student information.

Requirements

• The function should extract and return:

o Full Name

o Branch

o SGPA

Expected Output

A reusable Python function that correctly navigates and extracts values from nested dictionaries based on the provided examples

```
reate a function that parses a nested dictionary representing student information.
Requirements :
- The function should extract and return:
  Full Name, Branch, SGPA

Example 1:
{
    "name": "John Doe",
    "age": 20,
    "branch": "Computer Science",
    "academic": {
        "sgpa": 8.5,
        "year": 3
    }
}

Example 2:
{
    "name": "Jane Smith",
    "age": 22,
    "academic": {
        "sgpa": 9.1,
        "year": 4
    }
}
'''

def extract_student_info(student_dict):
    full_name = student_dict.get("name", "N/A")
    branch = student_dict.get("branch", "N/A")
    sgpa = student_dict.get("academic", {}).get("sgpa", "N/A")

    return {
        "Full Name": full_name,
        "Branch": branch,
        "SGPA": sgpa
    }

# Example usage
student_info = {
    "name": "John Doe",
    "age": 20,
    "branch": "Computer Science",
    "academic": {
        "sgpa": 8.5,
        "year": 3
    }
}

extracted_info = extract_student_info(student_info)
print(extracted_info)
```

output:

```
{'Full Name': 'John Doe', 'Branch': 'Computer Science', 'SGPA': 8.5}
```

Task Description #10 (Loops – Perfect Numbers in a Range)

Task: Generate a function using AI that displays all Perfect Numbers within a user-specified range (e.g., 1 to 1000).

Instructions:

• A Perfect Number is a positive integer equal to the sum of its proper divisors (excluding itself).

o Example: 6 = 1 + 2 + 3, 28 = 1 + 2 + 4 + 7 + 14.

• Use a for loop to find divisors of each number in the range.

• Validate correctness with known Perfect Numbers (6, 28, 496…).

• Ask AI to regenerate an optimized version (using divisor check only up to √n).

Expected Output #12:

• Python program that lists Perfect Numbers in the given range.

• Optimized version with explanation.

```
 Task 10:

''
enerate a function using AI that displays all Perfect Numbers within a user-specified range (e.g., 1 to 1000).

nstructions:
 A Perfect Number is a positive integer equal to the sum of its proper divisors (excluding itself).
   o Example: 6 = 1 + 2 + 3, 28 = 1 + 2 + 4 + 7 + 14.
 Use a for loop to find divisors of each number in the range.
 Validate correctness with known Perfect Numbers (6, 28, 496…).
 Ask AI to regenerate an optimized version (using divisor check only up to √n)
''
mport math

ef is_perfect(num):
   if num < 2:
       return False
   sum_of_divisors = 1  # 1 is a proper divisor of all integers > 1
   for i in range(2, int(math.sqrt(num)) + 1):
       if num % i == 0:
           sum_of_divisors += i
           if i != num // i:
               sum_of_divisors += num // i
   return sum_of_divisors == num
erfect_numbers = []
or i in range(1, 1001):
   if is_perfect(i):
       perfect_numbers.append(i)
rint("Perfect numbers between 1 and 1000 are:", perfect_numbers)
 Output: Perfect numbers between 1 and 1000 are: [6, 28
 Optimized version using divisor check only up to √n
ef is_perfect_optimized(num):
   if num < 2:
       return False
   sum_of_divisors = 1
   for i in range(2, int(math.sqrt(num)) + 1):
       if num % i == 0:
           sum_of_divisors += i
           if i != num // i:
               sum_of_divisors += num // i
   return sum_of_divisors == num
erfect_numbers_optimized = [num for num in range(1, 1001) if is_perfect_optimized(num)]
rint("Optimized Perfect numbers between 1 and 1000 are:", perfect_numbers_optimized)
 Output: Optimized Perfect numbers between 1 and 1000 are: [6, 28, 496]
 The optimized version improves efficiency by limiting the divisor check to the square root of the number, reducing the number of iterations nee
 Optimized version using divisor check only up to √n
```

output:

```
Perfect numbers between 1 and 1000 are: [6, 28, 496]
Optimized Perfect numbers between 1 and 1000 are: [6, 28, 496]
```